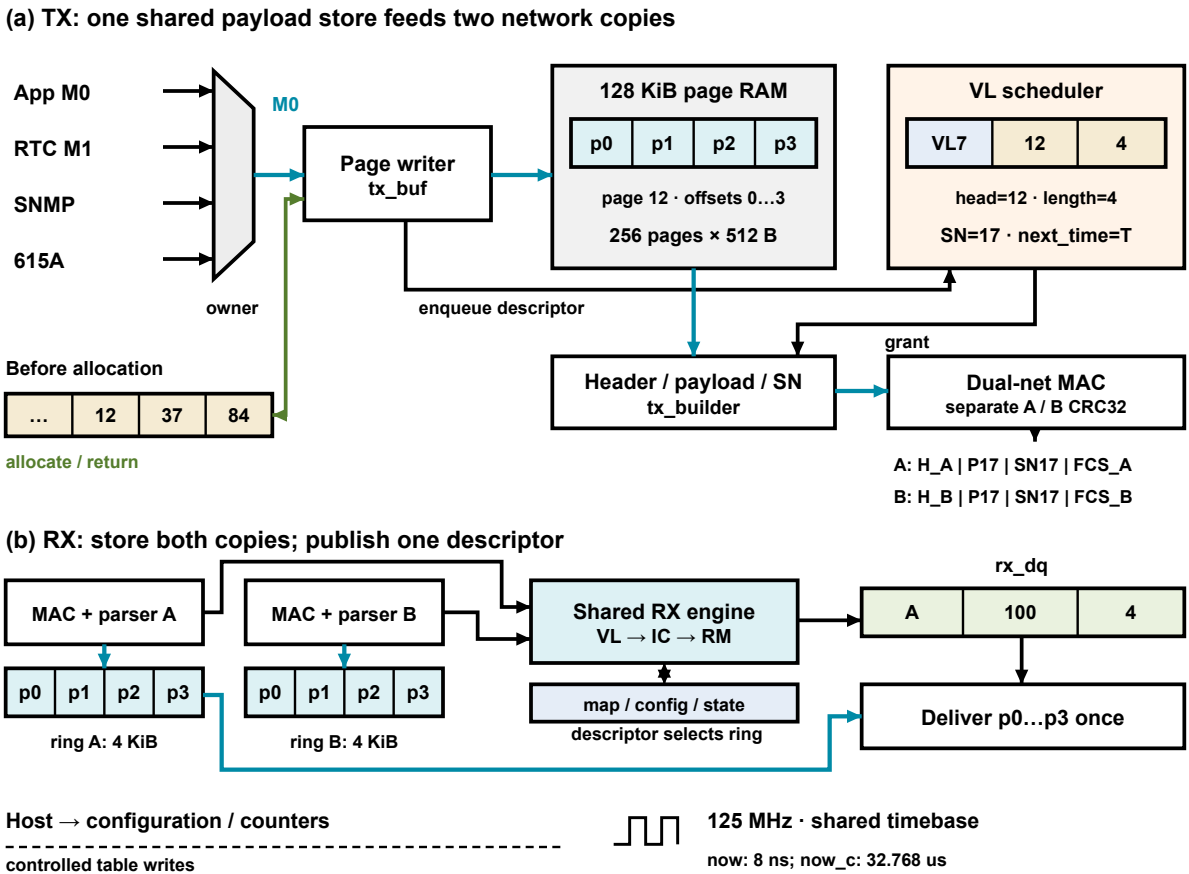
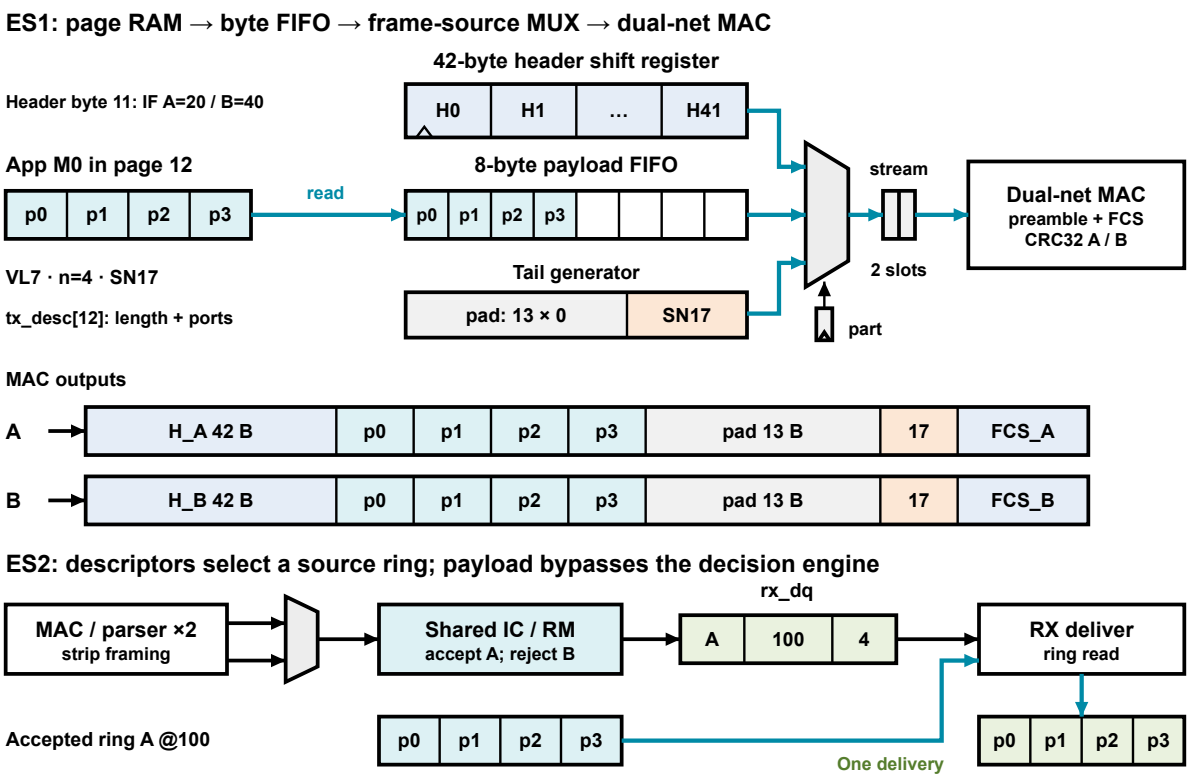
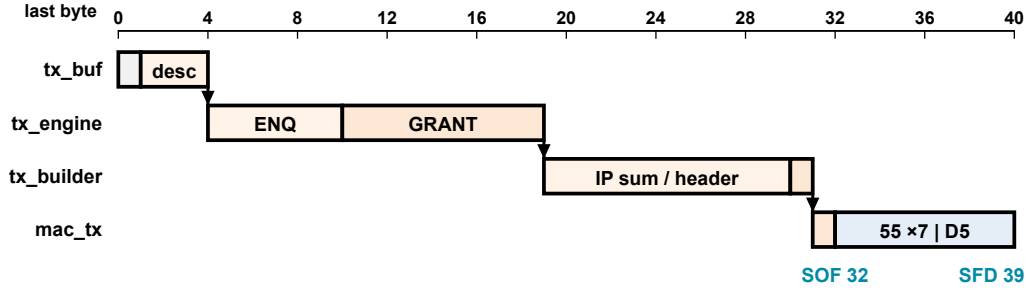
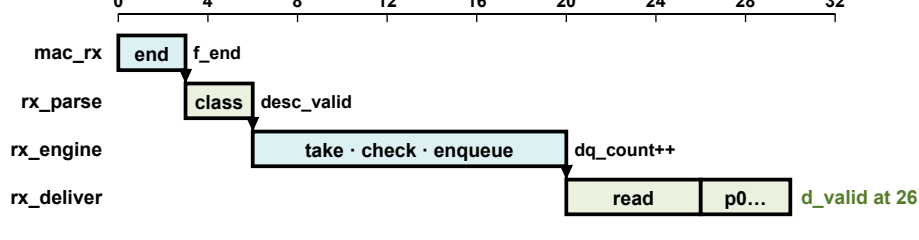

(a) TX: one shared payload store feeds two network copies
App M0
RTC M1
SNMP
615A
M0
owner
Page writer
tx_buf
128 KiB page RAM
p0
p1
p2
p3
page 12 · offsets 0…3
256 pages × 512 B
VL scheduler
VL7
12
4
head=12 · length=4
SN=17 · next_time=T
enqueue descriptor
grant
Before allocation
…
12
37
84
allocate / return
Header / payload / SN
tx_builder
Dual-net MAC
separate A / B CRC32
A: H_A | P17 | SN17 | FCS_A
B: H_B | P17 | SN17 | FCS_B
(b) RX: store both copies; publish one descriptor
MAC + parser A
MAC + parser B
Shared RX engine
VL → IC → RM
rx_dq
A
100
4
p0
p1
p2
p3
ring A: 4 KiB
ring B: 4 KiB
map / config / state
descriptor selects ring
Deliver p0…p3 once
Host → configuration / counters
controlled table writes
125 MHz · shared timebase
now: 8 ns; now_c: 32.768 us

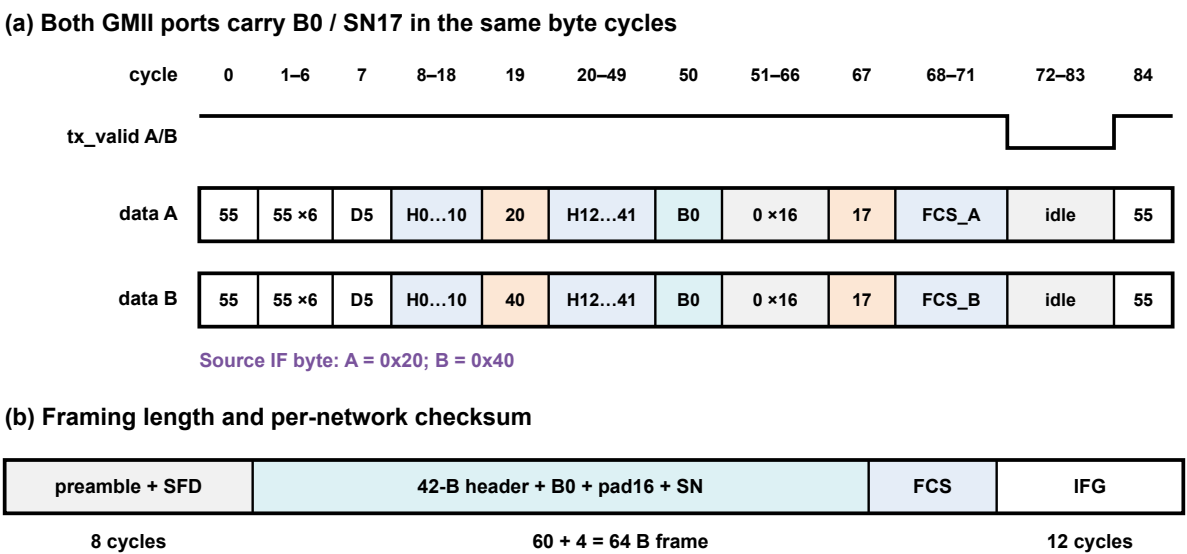

ES1: page RAM → byte FIFO → frame-source MUX → dual-net MAC
42-byte header shift register
Header byte 11: IF A=20 / B=40
H0
H1
…
H41
App M0 in page 12
8-byte payload FIFO
stream
Dual-net MAC
preamble + FCS
CRC32 A / B
p0
p1
p2
p3
read
p0
p1
p2
p3
VL7 · n=4 · SN17
Tail generator
2 slots
tx_desc[12]: length + ports
pad: 13 × 0
SN17
part
MAC outputs
A
H_A 42 B
p0
p1
p2
p3
pad 13 B
17
FCS_A
B
H_B 42 B
p0
p1
p2
p3
pad 13 B
17
FCS_B
ES2: descriptors select a source ring; payload bypasses the decision engine
rx_dq
MAC / parser ×2
strip framing
Shared IC / RM
accept A; reject B
A
100
4
RX deliver
ring read
Accepted ring A @100
p0
p1
p2
p3
p0
p1
p2
p3
One delivery

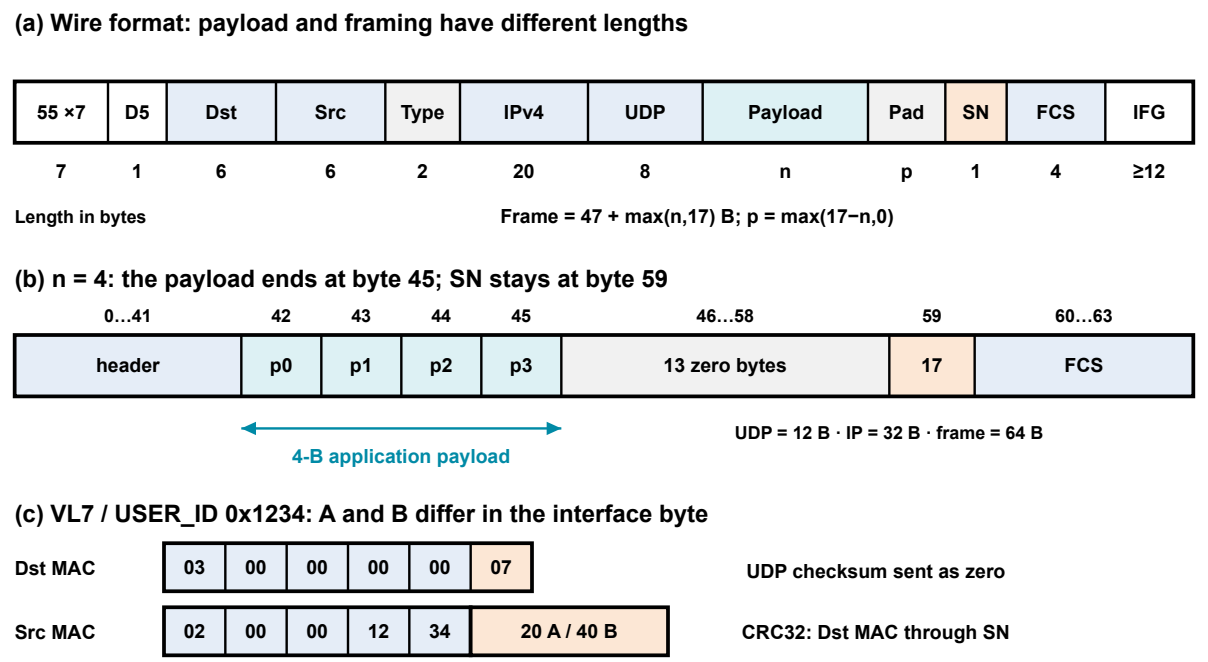
(a) TX: from the last application byte to the SFD byte

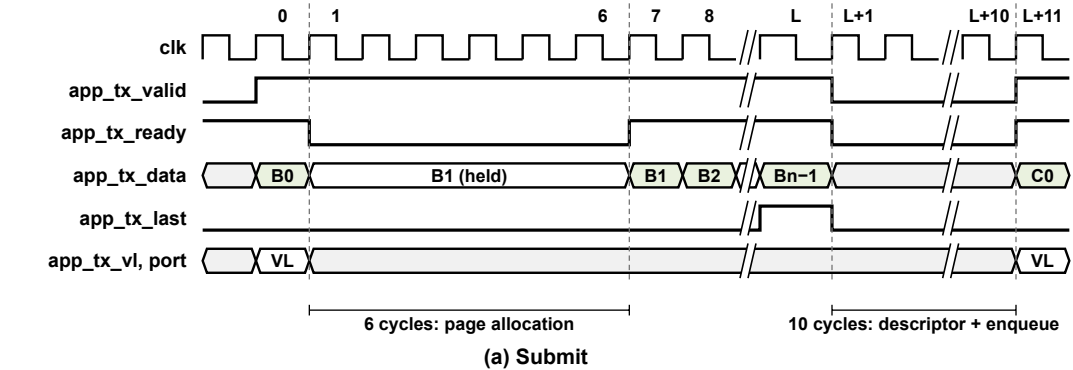


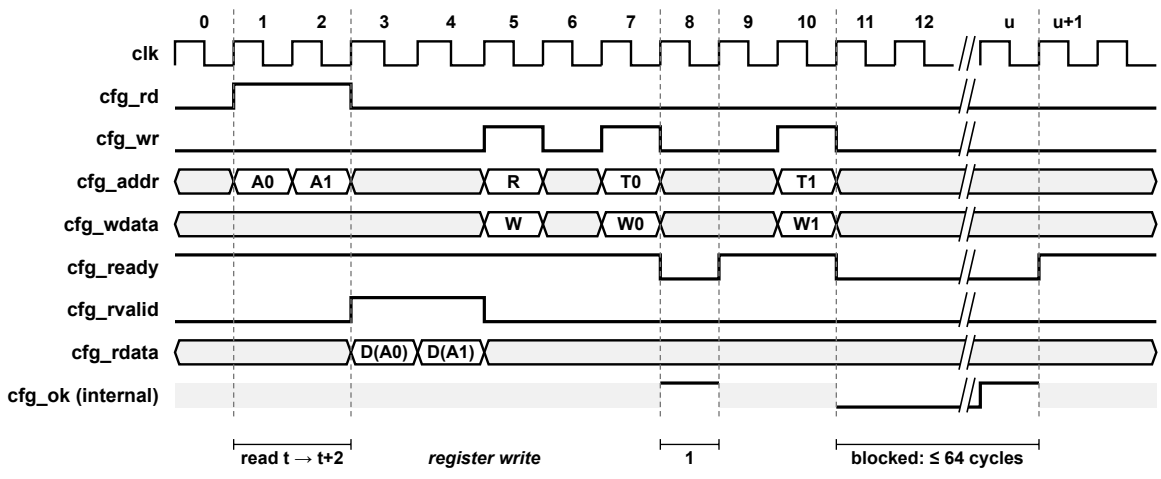
(b) RX: from the last FCS byte to the first delivered payload byte




(a) Both GMII ports carry B0 / SN17 in the same byte cycles
cycle
0
1–6
7
8–18
19
20–49
50
51–66
67
68–71
72–83
84
tx_valid A/B
data A
55
55 ×6
D5
H0…10
20
H12…41
B0
0 ×16
17
FCS_A
idle
55
data B
55
55 ×6
D5
H0…10
40
H12…41
B0
0 ×16
17
FCS_B
idle
55
Source IF byte: A = 0x20; B = 0x40
(b) Framing length and per-network checksum
preamble + SFD
42-B header + B0 + pad16 + SN
FCS
IFG
8 cycles
60 + 4 = 64 B frame
12 cycles

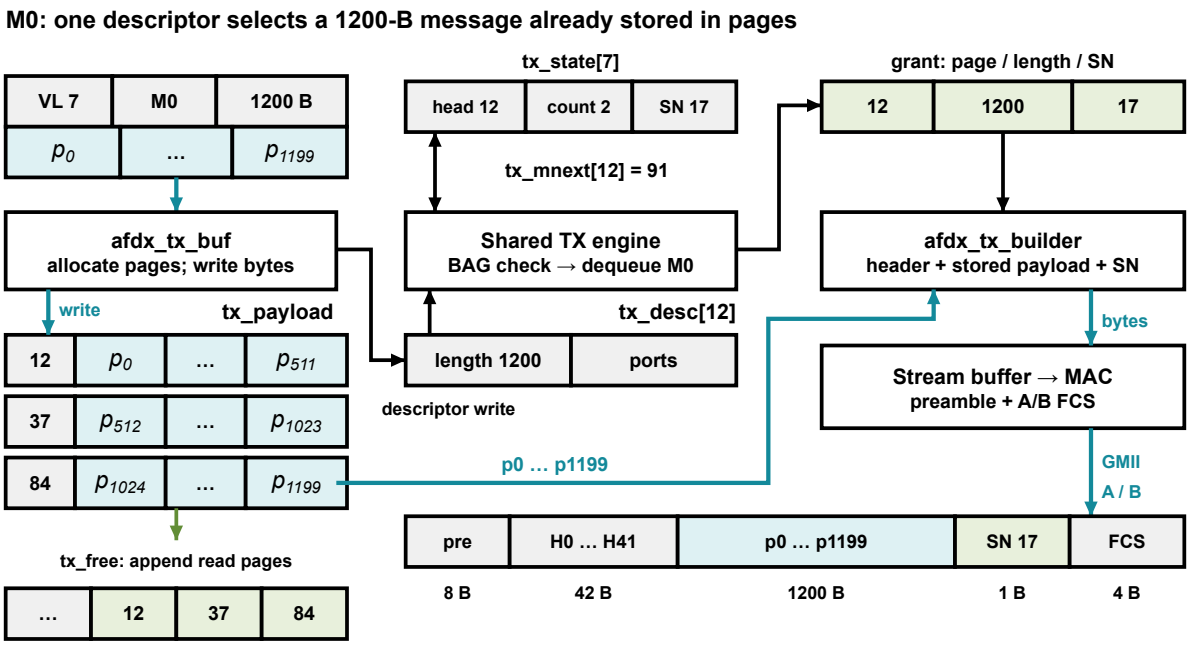

(a) Wire format: payload and framing have different lengths
55 ×7
D5
Dst
Src
Type
IPv4
UDP
Payload
Pad
SN
FCS
IFG
7
1
6
6
2
20
8
n
p
1
4
≥12
Length in bytes
Frame = 47 + max(n,17) B; p = max(17−n,0)
(b) n = 4: the payload ends at byte 45; SN stays at byte 59
0…41
42
43
44
45
46…58
59
60…63
header
p0
p1
p2
p3
13 zero bytes
17
FCS
4-B application payload
UDP = 12 B · IP = 32 B · frame = 64 B
(c) VL7 / USER_ID 0x1234: A and B differ in the interface byte
Dst MAC
03
00
00
00
00
07
UDP checksum sent as zero
Src MAC
02
00
00
12
34
20 A / 40 B
CRC32: Dst MAC through SN

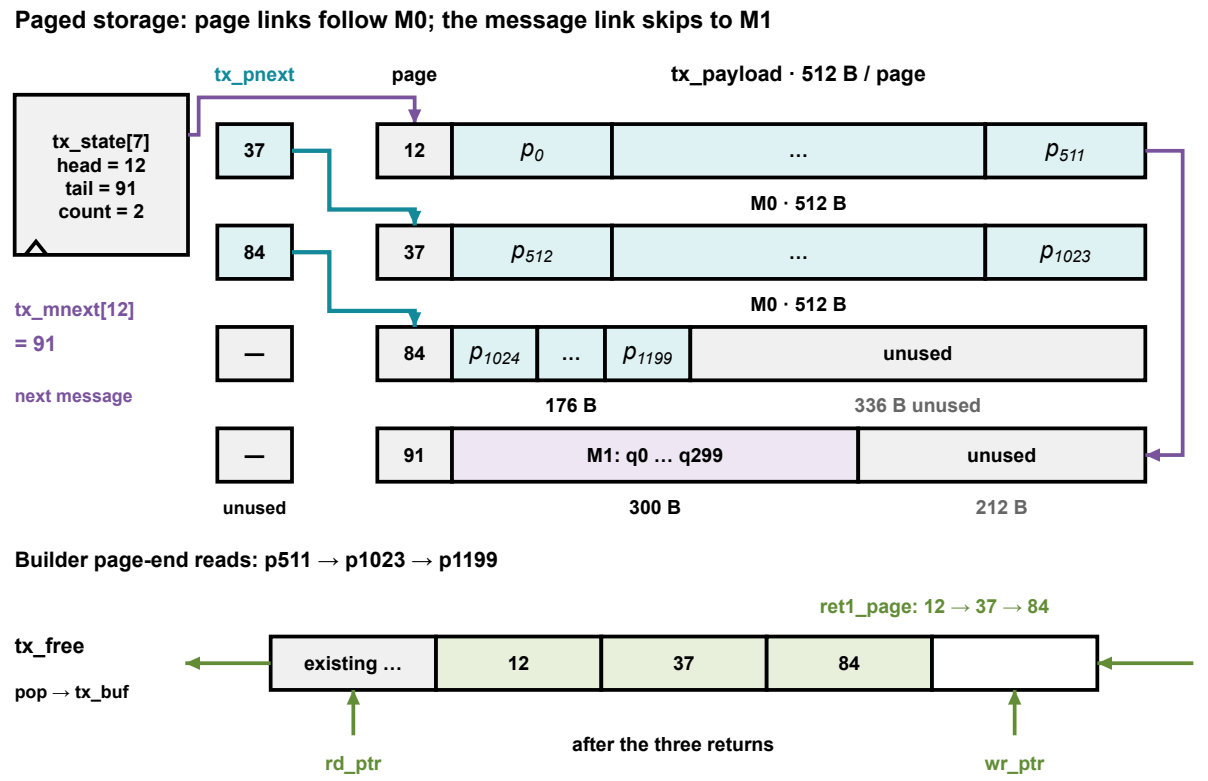

0
1
6
7
8
L
L+1
L+10
L+11
clk
app_tx_valid
app_tx_ready
app_tx_data
app_tx_last
app_tx_vl, port
B0
B1 (held)
B1
B2
Bn−1
C0
VL
VL
6 cycles: page allocation
10 cycles: descriptor + enqueue
(a) Submit
clk
app_rx_valid
app_rx_data
app_rx_last
app_rx_vl, port, net
D0
D1
D2
D3
Dn−1
constant
n consecutive cycles, no back-pressure
(b) Deliver
Same message, consecutive bytes
D0
D1
…
Dn−1
VL / ports / net stay fixed
d_last = 1 only on Dn−1
No receive-side ready signal



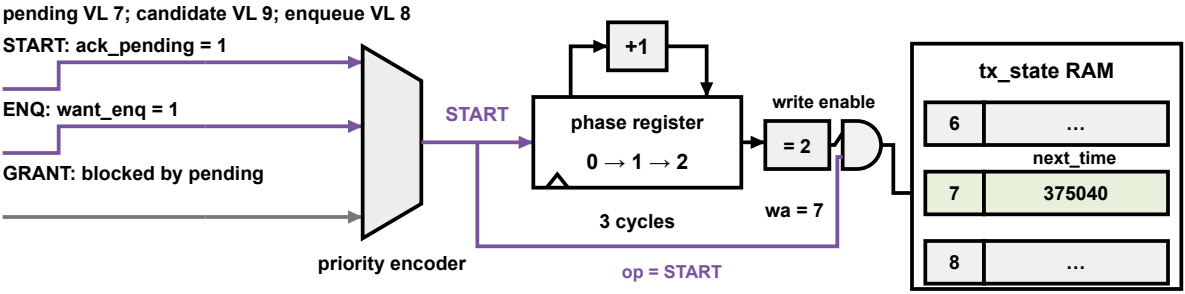
RX table write: the same T1 / W1 survives the blocked interval

accepted at 10	pending while cfg_ok = 0	permit at u	ready at u+1
{T1, W1}	{T1, W1, enable=1}	write T1 ← W1	enable=0


M0: one descriptor selects a 1200-B message already stored in pages
VL 7
M0
1200 B
p_0
…
p_{1199}
afdx_tx_buf
allocate pages; write bytes
write
tx_payload
12
p_0
…
p_{511}
37
p_{512}
…
p_{1023}
84
p_{1024}
…
p_{1199}
tx_free: append read pages
…
12
37
84
tx_state[7]
head 12
count 2
SN 17
tx_mnext[12] = 91
Shared TX engine
BAG check → dequeue M0
tx_desc[12]
length 1200
ports
descriptor write
p0 … p1199
grant: page / length / SN
12
1200
17
afdx_tx_builder
header + stored payload + SN
bytes
Stream buffer → MAC
preamble + A/B FCS
GMII
A / B
pre
H0 … H41
p0 … p1199
SN 17
FCS
8 B
42 B
1200 B
1 B
4 B
head: 12 → 91 count: 2 → 1 next SN: 17 → 18

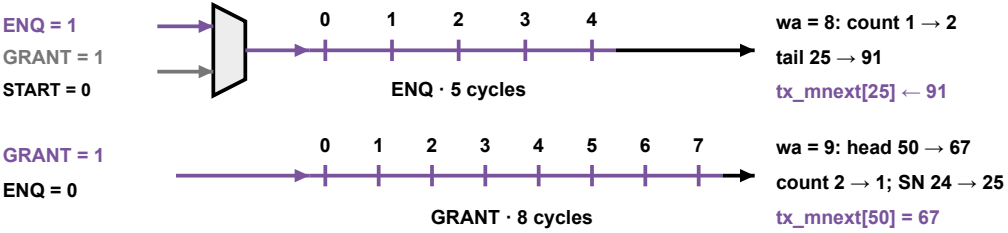

Paged storage: page links follow M0; the message link skips to M1
tx_pnext
page
tx_payload · 512 B / page
tx_state[7]
head = 12
tail = 91
count = 2
37
12
p_0
...
p_{511}
M0 · 512 B
84
37
p_{512}
...
p_{1023}
M0 · 512 B
tx_mnext[12]
= 91
—
84
p_{1024}
...
p_{1199}
unused
next message
176 B
336 B unused
—
91
M1: q0 … q299
unused
unused
300 B
212 B
Builder page-end reads: p511 → p1023 → p1199
ret1_page: 12 → 37 → 84
tx_free
pop → tx_buf
existing …
12
37
84
after the three returns
rd_ptr
wr_ptr

(a) START wins: priority logic drives one phase counter and one RAM write port



START writes 250040 + 125000; then clears pend_valid.

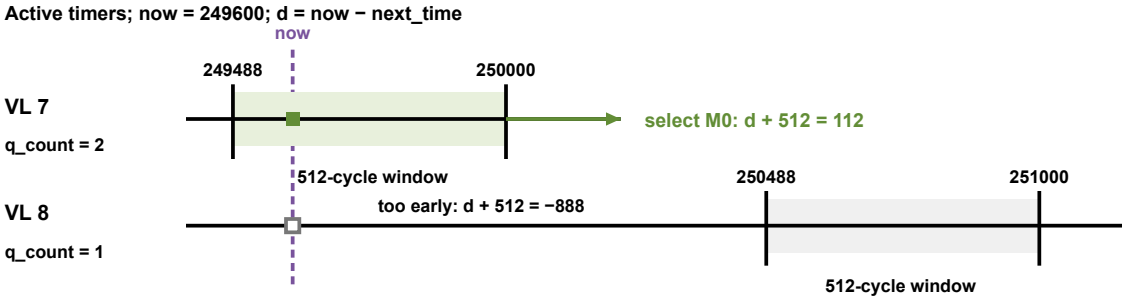
(b) Later IDLE decisions reuse the same counter and write port



GRANT sets pend_vl = 9. Only the selected operation can assert state_we.

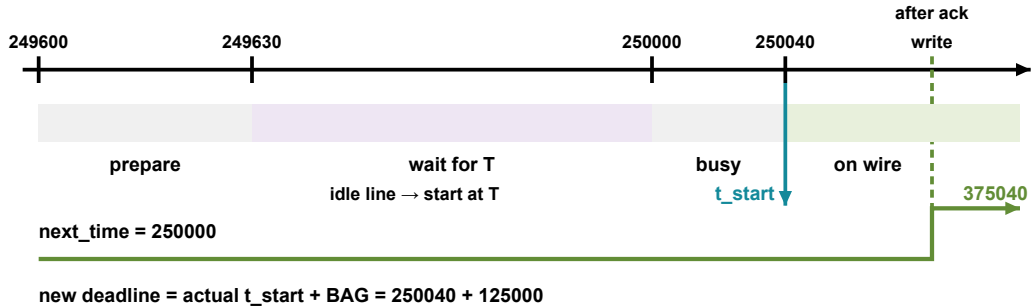
Counter-lane durations exclude the IDLE dispatch cycle.

Eligibility is a time window; pending temporarily excludes the selected VL

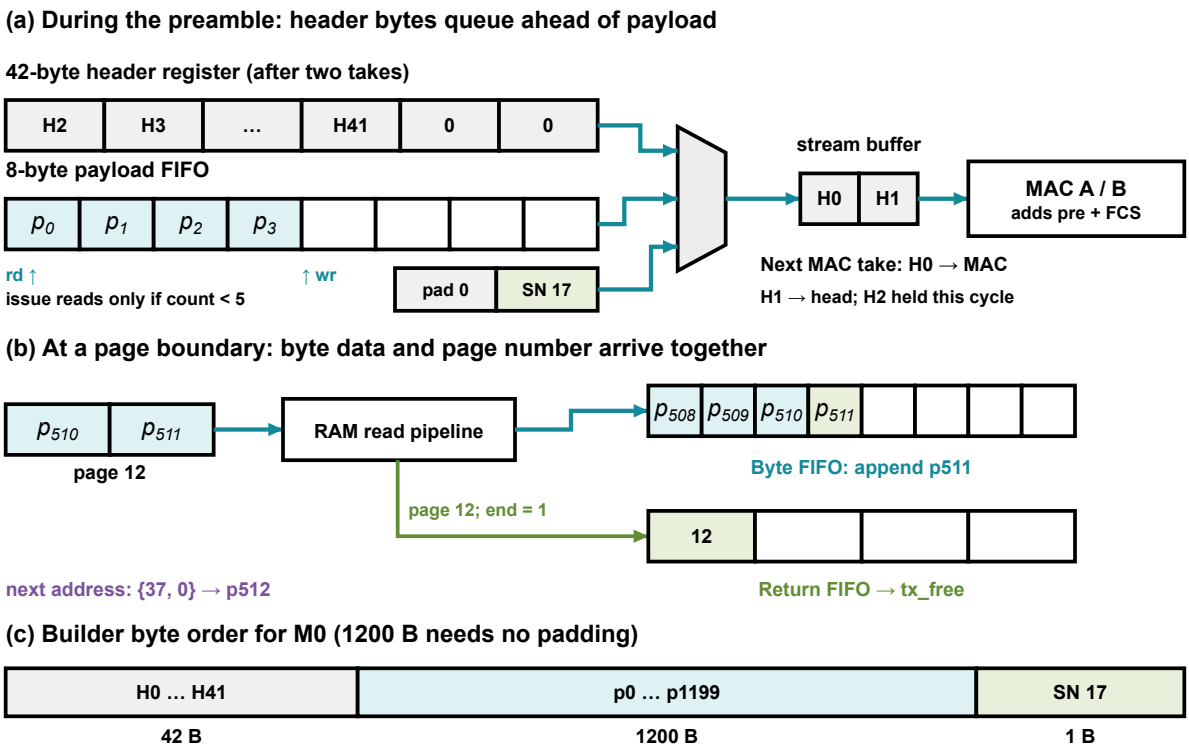


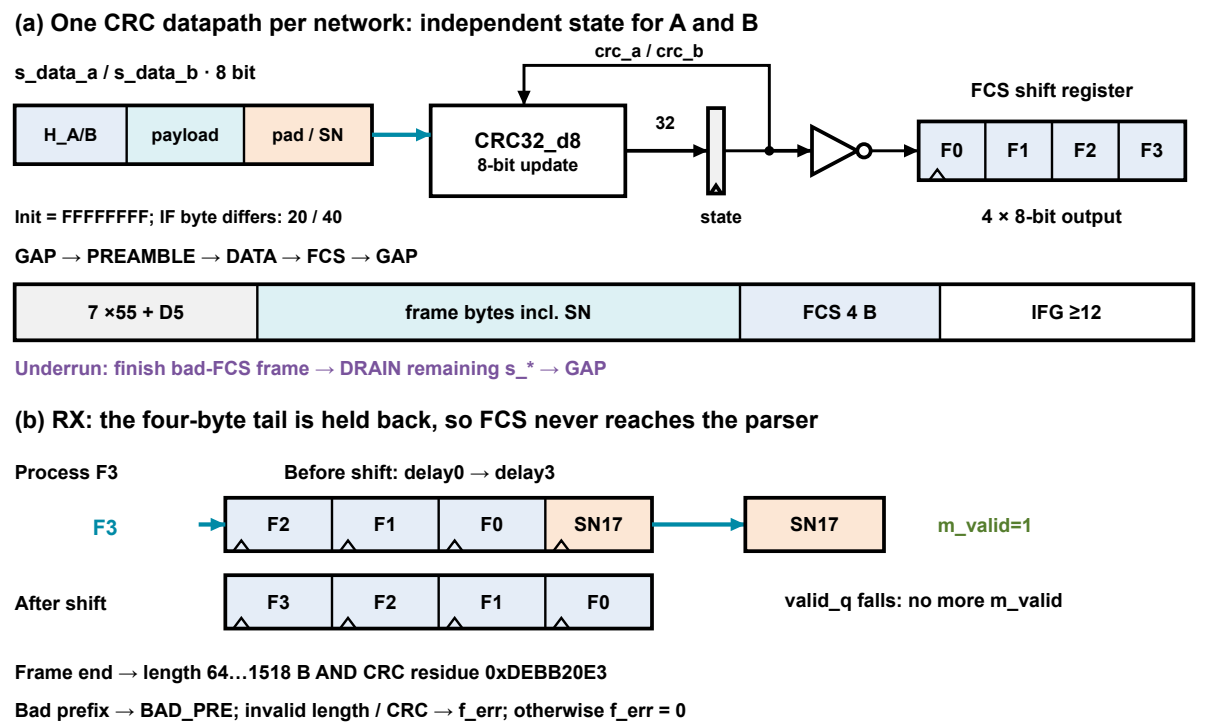
Grant M0: q_count 2 → 1; pend_vl = 7 blocks M1 until START clears pending.

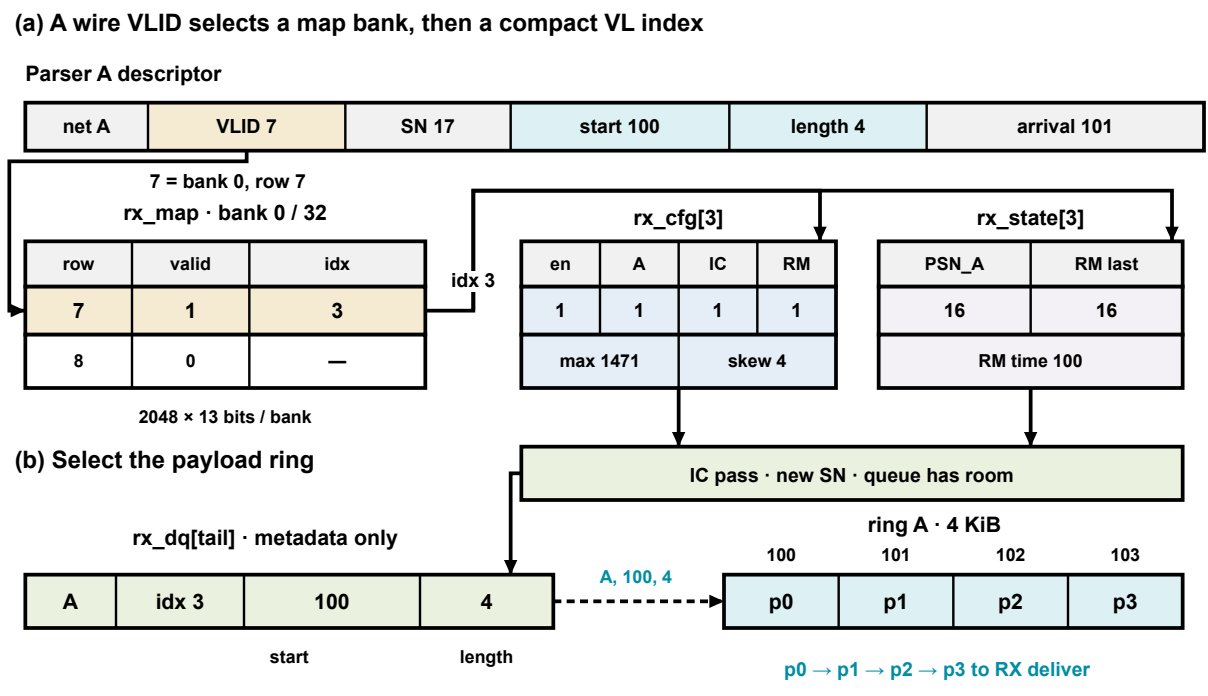
M0 timeline and the stored deadline (illustrative; lower axis not to scale)

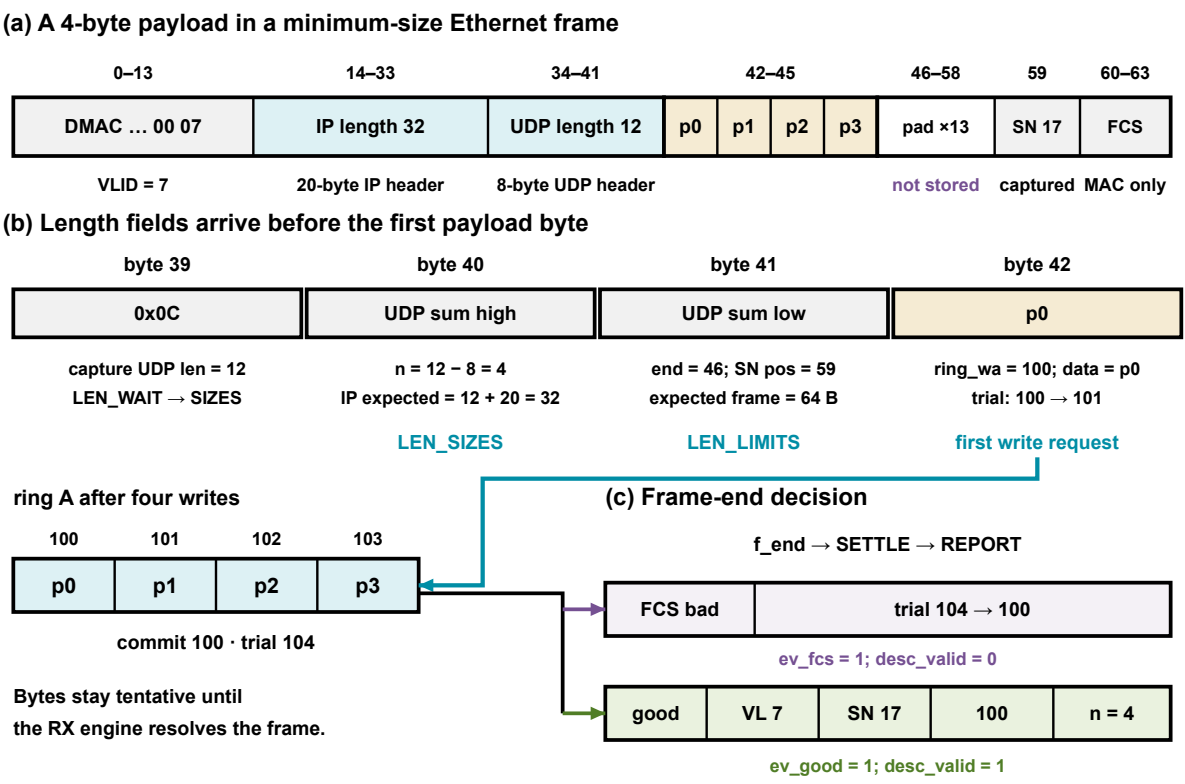


Prepared before T + idle line → start at T. timer_active = 0 → skip the T check.

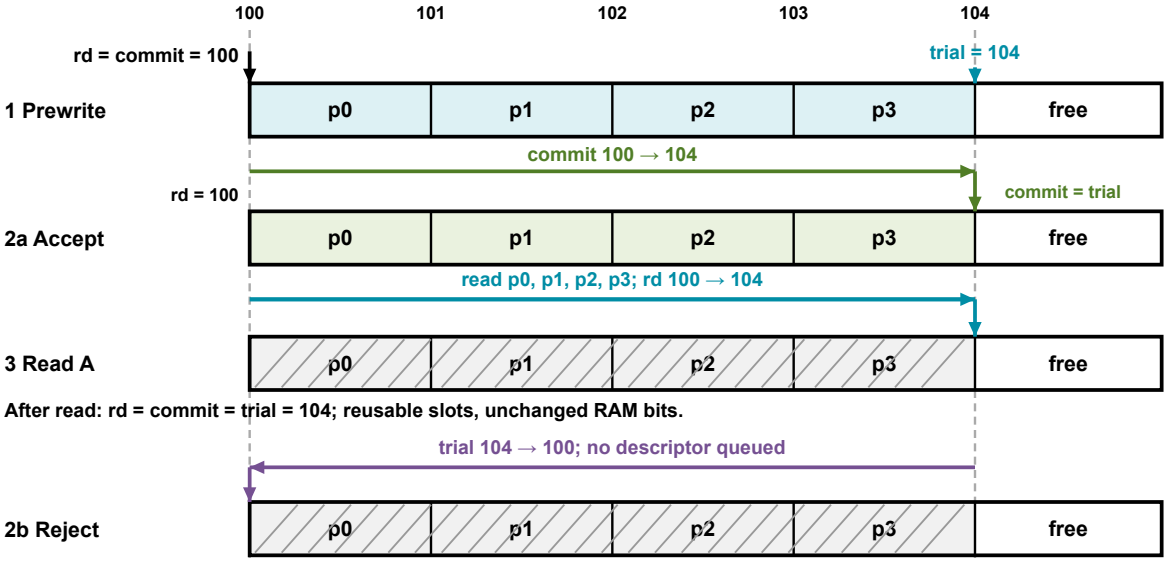

(a) During the preamble: header bytes queue ahead of payload
42-byte header register (after two takes)
H2
H3
…
H41
0
0
8-byte payload FIFO
p0
p1
p2
p3
rd ↑
issue reads only if count < 5
↑ wr
pad 0
SN 17
stream buffer
H0
H1
MAC A / B
adds pre + FCS
Next MAC take: H0 → MAC
H1 → head; H2 held this cycle
(b) At a page boundary: byte data and page number arrive together
p510
p511
page 12
RAM read pipeline
page 12; end = 1
next address: {37, 0} → p512
p508
p509
p510
p511
Byte FIFO: append p511
12
Return FIFO → tx_free
(c) Builder byte order for M0 (1200 B needs no padding)
H0 … H41
p0 … p1199
SN 17
42 B
1200 B
1 B


(a) One CRC datapath per network: independent state for A and B
s_data_a / s_data_b · 8 bit
crc_a / crc_b
FCS shift register
H_A/B
payload
pad / SN
CRC32_d8
8-bit update
32
F0
F1
F2
F3
Init = FFFFFFFF; IF byte differs: 20 / 40
state
4 × 8-bit output
GAP → PREAMBLE → DATA → FCS → GAP
7 ×55 + D5
frame bytes incl. SN
FCS 4 B
IFG ≥12
Underrun: finish bad-FCS frame → DRAIN remaining s_* → GAP
(b) RX: the four-byte tail is held back, so FCS never reaches the parser
Process F3
Before shift: delay0 → delay3
F3
F2
F1
F0
SN17
SN17
m_valid=1
After shift
F3
F2
F1
F0
valid_q falls: no more m_valid
Frame end → length 64…1518 B AND CRC residue 0xDEBB20E3
Bad prefix → BAD_PRE; invalid length / CRC → f_err; otherwise f_err = 0

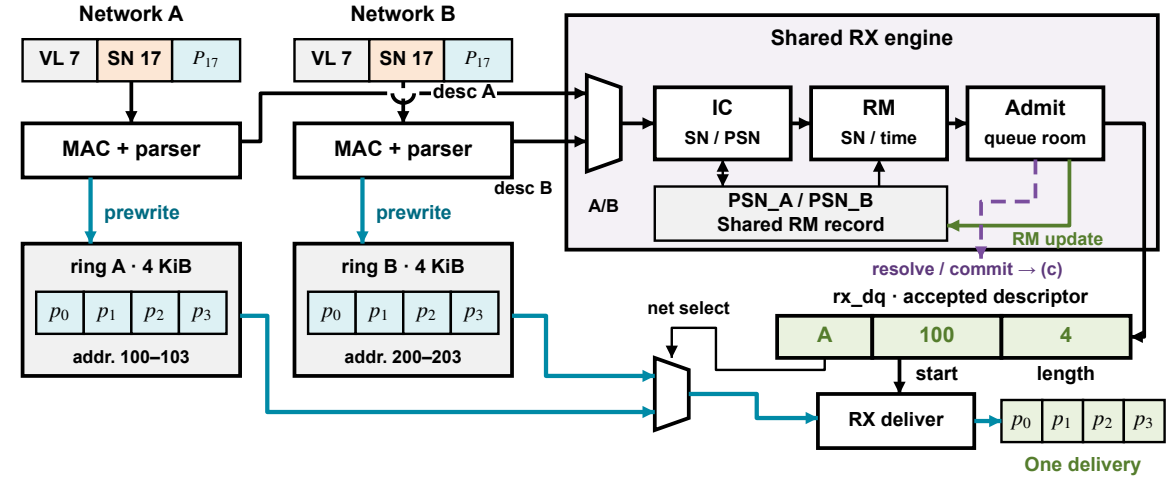

(a) A wire VLID selects a map bank, then a compact VL index
Parser A descriptor
net A
VLID 7
SN 17
start 100
length 4
arrival 101
7 = bank 0, row 7
rx_map · bank 0 / 32
row
valid
idx
7
1
3
8
0
—
2048 × 13 bits / bank
idx 3
rx_cfg[3]
en
A
IC
RM
1
1
1
1
max 1471
skew 4
rx_state[3]
PSN_A
RM last
16
16
RM time 100
(b) Select the payload ring
IC pass · new SN · queue has room
rx_dq[tail] · metadata only
A
idx 3
100
4
start
length
A, 100, 4
ring A · 4 KiB
100
101
102
103
p0
p1
p2
p3
p0 → p1 → p2 → p3 to RX deliver


(a) A 4-byte payload in a minimum-size Ethernet frame
0–13
14–33
34–41
42–45
46–58
59
60–63
DMAC … 00 07
IP length 32
UDP length 12
p0
p1
p2
p3
pad ×13
SN 17
FCS
VLID = 7
20-byte IP header
8-byte UDP header
not stored
captured
MAC only
(b) Length fields arrive before the first payload byte
byte 39
byte 40
byte 41
byte 42
0x0C
UDP sum high
UDP sum low
p0
capture UDP len = 12
LEN_WAIT → SIZES
n = 12 − 8 = 4
IP expected = 12 + 20 = 32
end = 46; SN pos = 59
expected frame = 64 B
ring_wa = 100; data = p0
trial: 100 → 101
LEN_SIZES
LEN_LIMITS
first write request
ring A after four writes
100
101
102
103
p0
p1
p2
p3
commit 100 · trial 104
Bytes stay tentative until
the RX engine resolves the frame.
(c) Frame-end decision
f_end → SETTLE → REPORT
FCS bad
trial 104 → 100
ev_fcs = 1; desc_valid = 0
good
VL 7
SN 17
100
n = 4
ev_good = 1; desc_valid = 1

A four-byte trial: publish, consume, or release the same addresses



(a) Dual-network reception



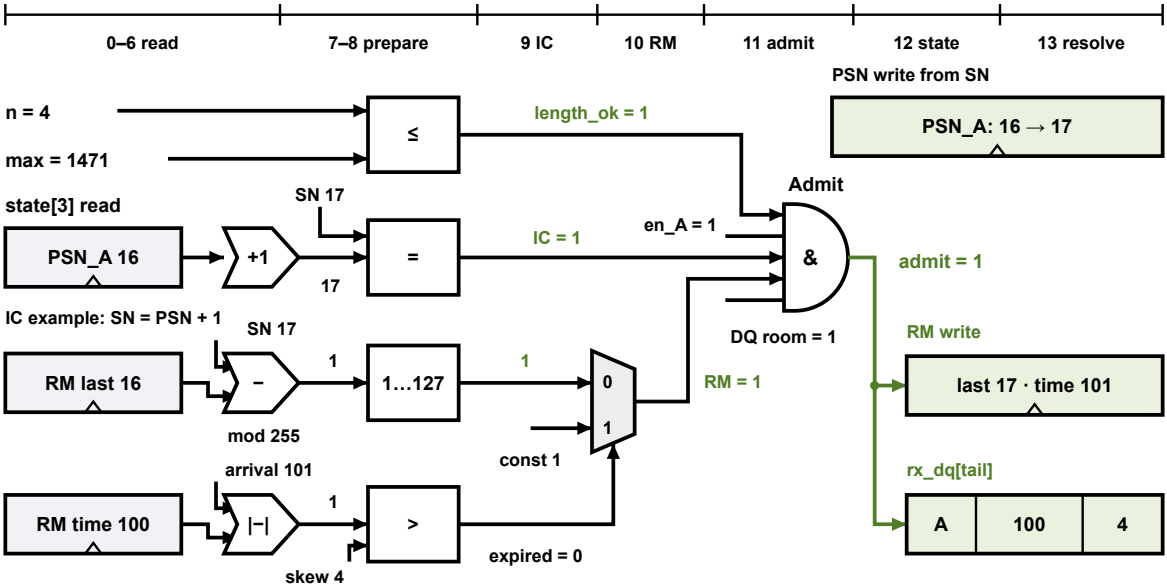
(b) Same VL / SN: process A, then B

Net	SN	IC: local PSN=16	RM record	RM result	commit	Queue write
A	17	Pass	16 → 17	New frame	1	{A, 100, 4}
B	17	Pass	17 → 17	Duplicate	0	—

(c) Commit and rollback



SN 17: field comparisons and gated state updates

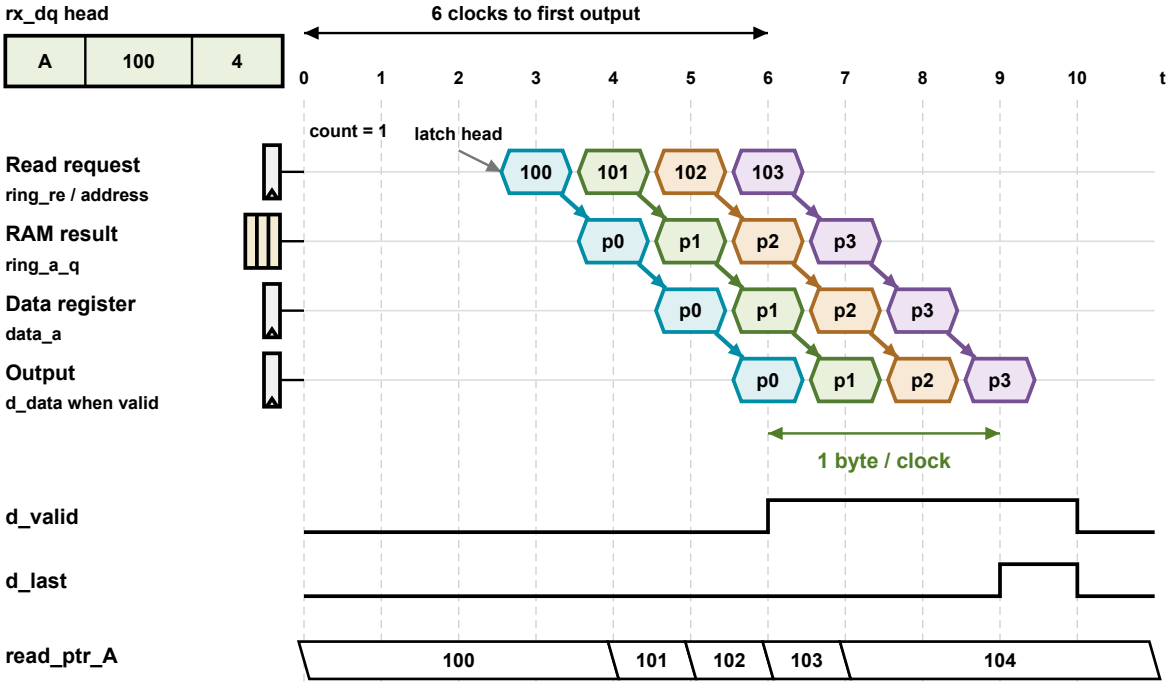


IC-enabled PSN follows SN even on IC failure; RM and DQ writes require admit.

phase 13: admit → commit=1; rejected → commit=0 (trial rollback).

AGE 0–3 clears expired RM valid; any pending descriptor returns the engine to IDLE.

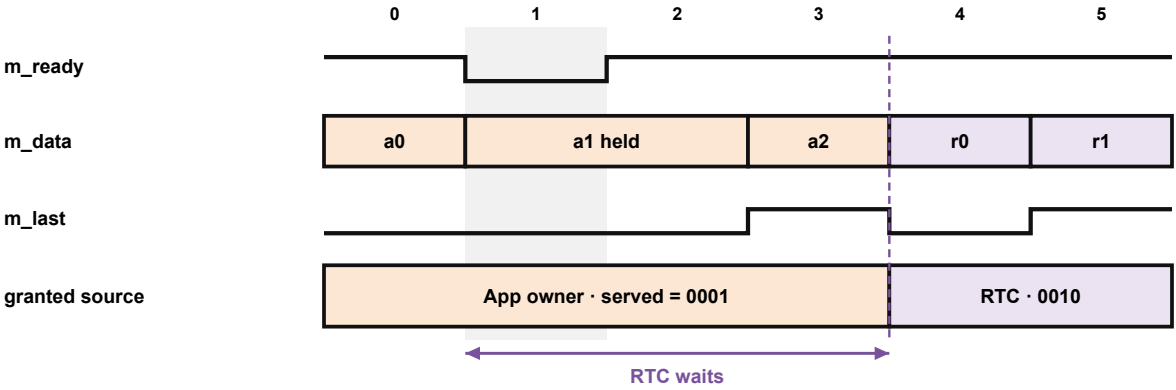
A four-byte read: startup latency, then one output byte per clock



t9: p3 and d_last coincide; dq_req re-arms. Metadata stays fixed; ring B is not read.

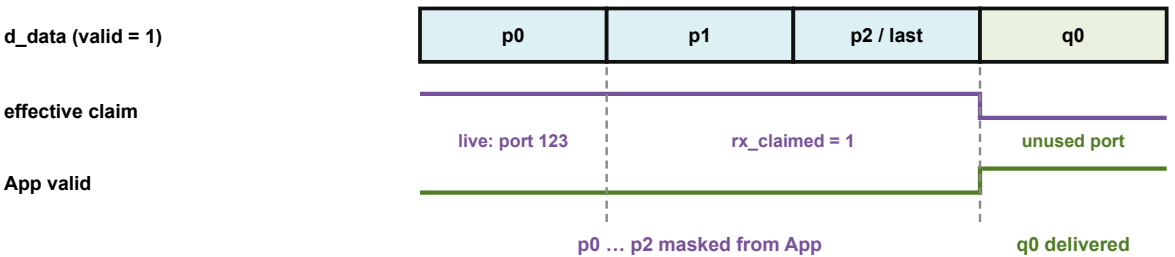
(a) TX arbitration: hold the message owner through back-pressure

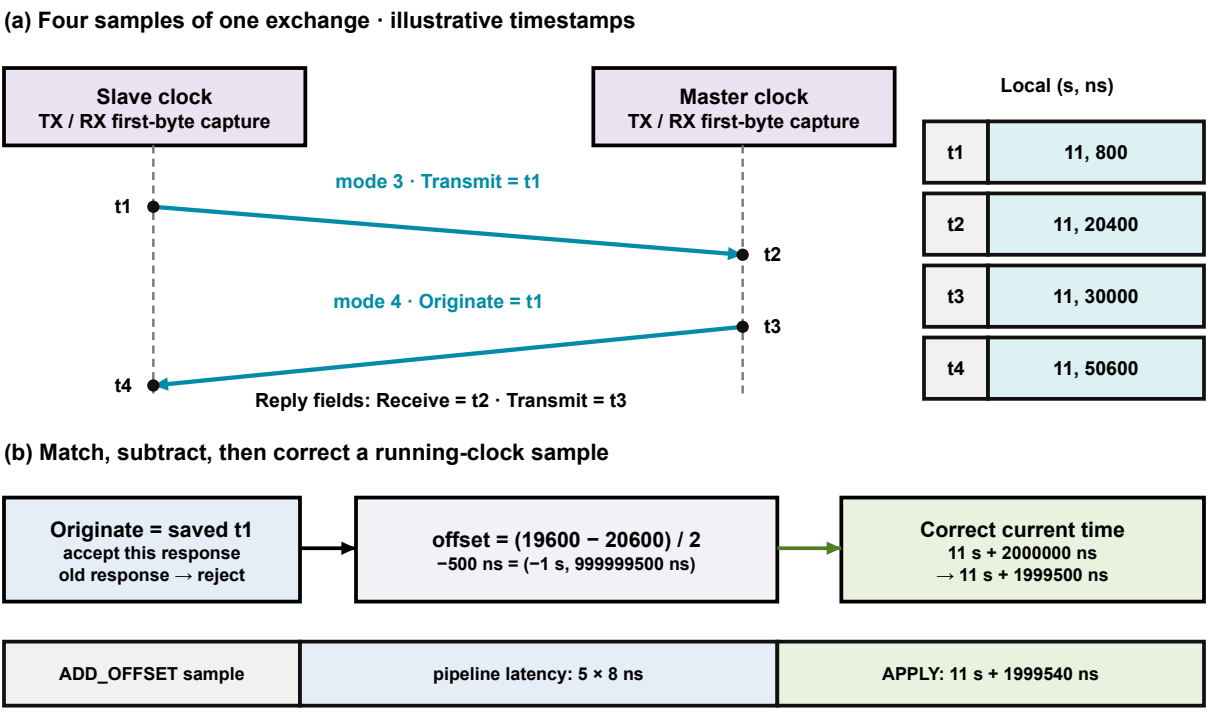
App: a0 a1 a2(last) RTC: r0 r1(last), requesting from cycle 1

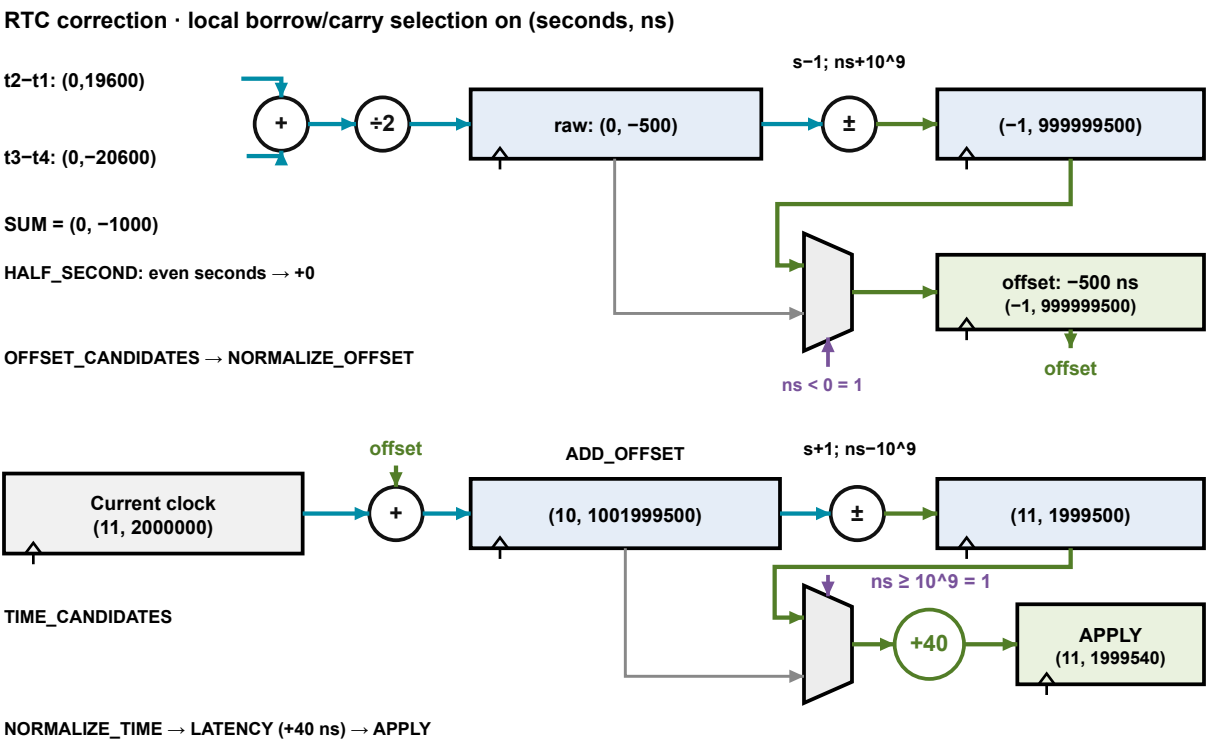


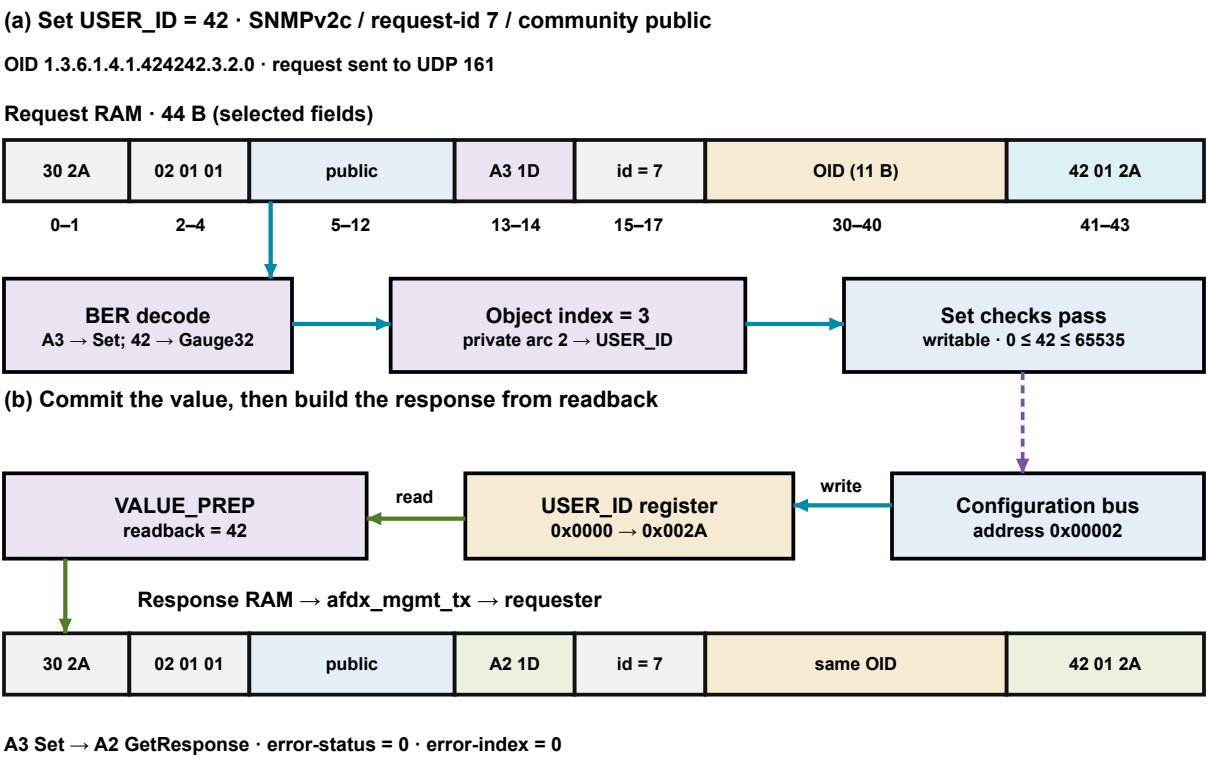
Cycle 1: a1 stalls. End of cycle 3: a2/last handshakes; RTC starts at cycle 4.

(b) RX port claim: a live first-byte decision becomes a held mask



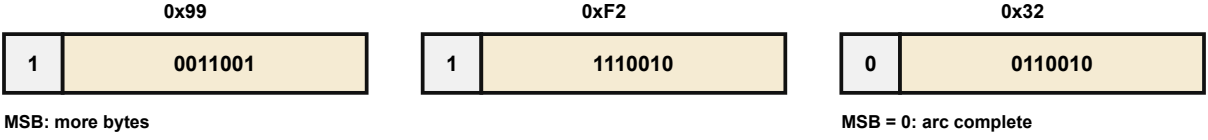

(a) Four samples of one exchange · illustrative timestamps
Slave clock
TX / RX first-byte capture
Master clock
TX / RX first-byte capture
mode 3 · Transmit = t1
t1
t2
mode 4 · Originate = t1
t3
t4
Reply fields: Receive = t2 · Transmit = t3
Local (s, ns)
t1
11, 800
t2
11, 20400
t3
11, 30000
t4
11, 50600
(b) Match, subtract, then correct a running-clock sample
Originate = saved t1
accept this response
old response → reject
offset = (19600 − 20600) / 2
−500 ns = (−1 s, 999999500 ns)
Correct current time
11 s + 2000000 ns
→ 11 s + 1999500 ns
ADD_OFFSET sample
pipeline latency: 5 × 8 ns
APPLY: 11 s + 1999540 ns

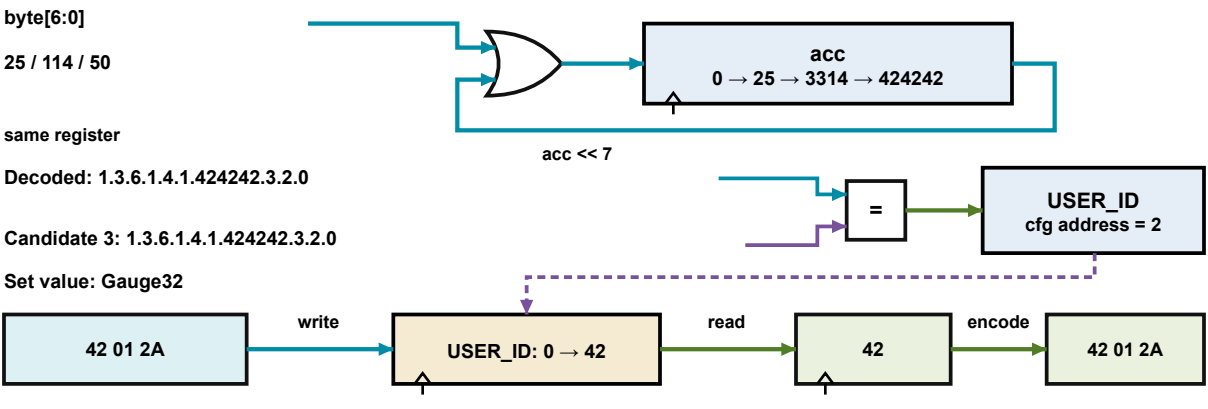

RTC correction · local borrow/carry selection on (seconds, ns)
t2−t1: (0,19600)
t3−t4: (0,−20600)
SUM = (0, −1000)
HALF_SECOND: even seconds → +0
OFFSET_CANDIDATES → NORMALIZE_OFFSET
+
÷2
raw: (0, −500)
s−1; ns+10^9
±
(−1, 999999500)
offset: −500 ns
(−1, 999999500)
offset
ns < 0 = 1
offset
Current clock
(11, 2000000)
+
ADD_OFFSET
(10, 1001999500)
s+1; ns−10^9
±
(11, 1999500)
ns ≥ 10^9 = 1
TIME_CANDIDATES
+40
APPLY
(11, 1999540)
NORMALIZE_TIME → LATENCY (+40 ns) → APPLY


(a) Set USER_ID = 42 · SNMPv2c / request-id 7 / community public
OID 1.3.6.1.4.1.424242.3.2.0 · request sent to UDP 161
Request RAM · 44 B (selected fields)
30 2A
02 01 01
public
A3 1D
id = 7
OID (11 B)
42 01 2A
0–1
2–4
5–12
13–14
15–17
30–40
41–43
BER decode
A3 → Set; 42 → Gauge32
Object index = 3
private arc 2 → USER_ID
Set checks pass
writable · 0 ≤ 42 ≤ 65535
(b) Commit the value, then build the response from readback
VALUE_PREP
readback = 42
read
USER_ID register
0x0000 → 0x002A
write
Configuration bus
address 0x00002
Response RAM → afdx_mgmt_tx → requester
30 2A
02 01 01
public
A2 1D
id = 7
same OID
42 01 2A
A3 Set → A2 GetResponse · error-status = 0 · error-index = 0

SNMP Set 42: decode the enterprise arc, select USER_ID, read it back

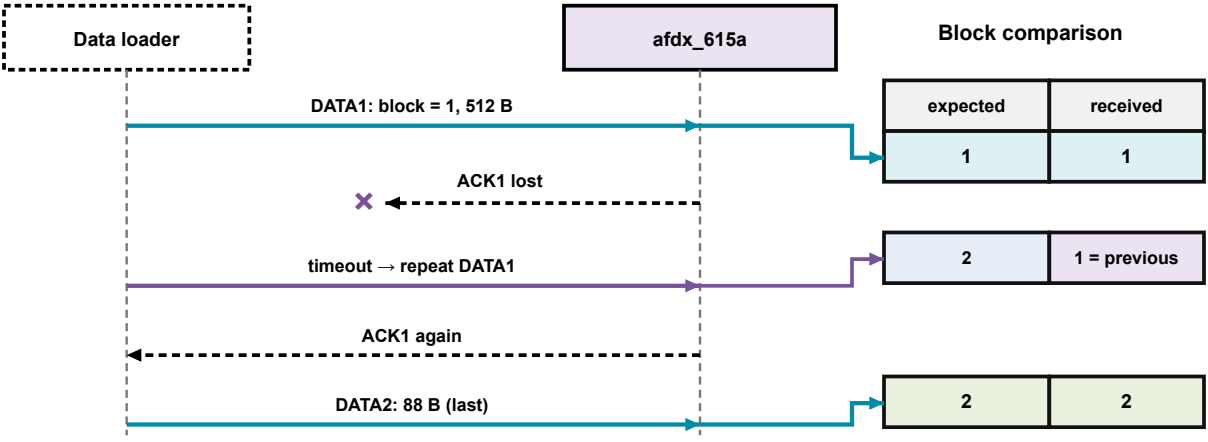
Request RAM[35:37]: enterprise arc in OID 1.3.6.1.4.1.424242.3.2.0





A2 GetResponse: request-id 7 · same OID · actual readback 42

(a) Lost ACK1: resend the block, never append it twice



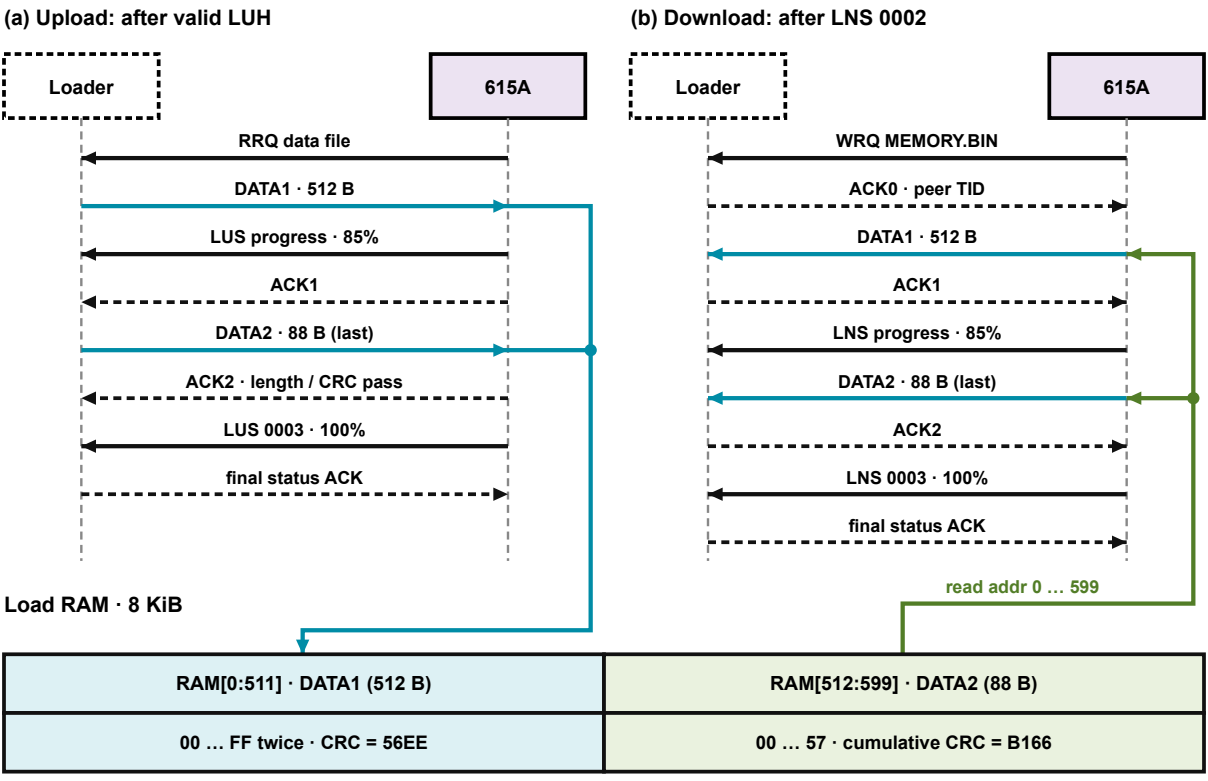
(b) DATA_DISPATCH selects copying or acknowledgment only

Arrival	FSM action	RAM write	loaded	CRC (hex)
DATA1	DATA_COPY	[0:511] ← 512 B	0 → 512	56EE
repeat DATA1	ACK → WAIT_PACKET	none	512 → 512	56EE
DATA2	COPY → file check	[512:599] ← 88 B	512 → 600	B166

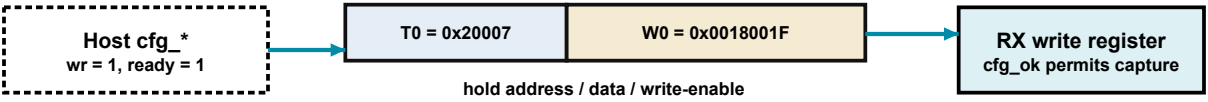
DATA1 progress status completes before ACK1; final length / CRC checks precede ACK2.

600-byte file · byte[i] = i mod 256 · expected CRC-16 = B166

Upload setup: LUI 0001 → initial LUS → LUR → LUS 0002 → LUH → LUS progress



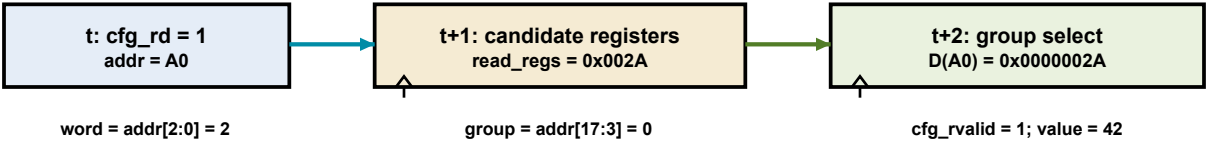
(a) One table write: T0 = rx_cfg[7] at 0x20007; W0 = 0x0018001F



Cycle / edge	t	t+1 …	u	u+1	u+2
bus ready (RX)					
held write	capture	T0, W0	T0, W0	clear	—
core cfg_ok					
RX RAM[7]	old	old	old	W0 written	W0
bus ready (TX)					

u: capture by core; TX comparison uses tx_cfg (e.g. 0x01038) and rearms ready one cycle later.

(b) Read A0 = 0x00002 after SNMP Set: USER_ID = 0x002A



Host and SNMP share this bus; same-address reads can be combined.